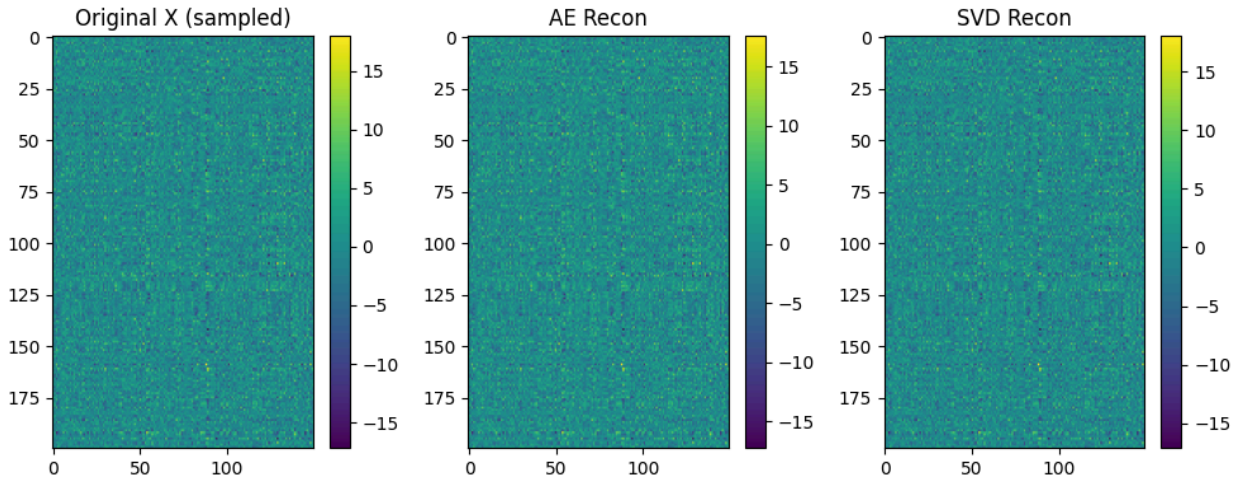
Assignment 5

```
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
import torch.optim as optim
import math
import random
import matplotlib.pyplot as plt

# Reproducibility
seed = 42
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)

# 1) Synthetic data: low-rank matrix + noise
m, n = 200, 150        # matrix size (rows, cols)
true_rank = 10         # true generative rank
target_rank = 10       # k used in reconstruction (bottleneck size)

# create low-rank matrix X0 = A B^T where A:(m x r), B:(n x r)
A = np.random.randn(m, true_rank)
B = np.random.randn(n, true_rank)
X0 = A.dot(B.T)                            # shape (m, n)
noise_level = 0.05
X = X0 + noise_level * np.random.randn(m, n)  # observed matrix

# Convert to torch tensors
X_torch = torch.tensor(X, dtype=torch.float32)

# We'll treat each row as a datapoint for the autoencoder (reconstruct rows)
dataset = TensorDataset(X_torch)
batch_size = 32
loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

# 2) Simple fully-connected autoencoder (rows -> latent -> rows)
class AE(nn.Module):
    def __init__(self, n_features, bottleneck):
        super().__init__()
        # Encoder: reduce to bottleneck
        self.enc = nn.Sequential(
            nn.Linear(n_features, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, bottleneck)  # linear bottleneck (like factors)
        )
        # Decoder: expand back to n_features
        self.dec = nn.Sequential(
            nn.Linear(bottleneck, 64),
            nn.ReLU(),
            nn.Linear(64, 128),
            nn.ReLU(),
            nn.Linear(128, n_features)  # final linear layer (reconstruction)
        )
```

```
    def forward(self, x):
        z = self.enc(x)
        out = self.dec(z)
        return out, z

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = AE(n_features=n, bottleneck=target_rank).to(device)

# 3) Training setup
lr = 1e-3
epochs = 400
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=1e-6)

# training loop
train_losses = []
for epoch in range(1, epochs+1):
    model.train()
    epoch_loss = 0.0
    for (batch,) in loader:
        batch = batch.to(device)
        recon, _ = model(batch)
        loss = criterion(recon, batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item() * batch.size(0)
    epoch_loss /= m
    train_losses.append(epoch_loss)
    if epoch % 50 == 0 or epoch == 1:
        print(f"Epoch {epoch:03d}, Train MSE: {epoch_loss:.6f}")

# 4) Get autoencoder reconstruction for the whole matrix
model.eval()
with torch.no_grad():
    X_t = X_torch.to(device)
    recon_full, latent = model(X_t)
    recon_np = recon_full.cpu().numpy()

# compute MSE of autoencoder reconstruction (row-wise)
mse_ae = np.mean((recon_np - X)**2)
print(f"\nAutoencoder reconstruction MSE: {mse_ae:.8f}")

# 5) Compute optimal rank-k SVD reconstruction using numpy
# We compute the rank-k truncated SVD of X (rows = m, cols = n)
U, s, Vt = np.linalg.svd(X, full_matrices=False)
k = target_rank
Uk = U[:, :k]
sk = s[:k]
Vtk = Vt[:k, :]
X_svd_k = Uk * sk @ Vtk   # broadcasting: Uk * sk = (m,k) * (k,) => (m,k)

mse_svd = np.mean((X_svd_k - X)**2)
print(f"NumPy truncated SVD (k={k}) reconstruction MSE: {mse_svd:.8f}")

# 6) Compare with true underlying low-rank X0 (if you want)
mse_true_vs_X = np.mean((X - X0)**2)
mse_ae_vs_X0 = np.mean((recon_np - X0)**2)
mse_svd_vs_X0 = np.mean((X_svd_k - X0)**2)
print(f"MSE noisy vs true X0: {mse_true_vs_X:.8f}")
print(f"Autoencoder vs true X0: {mse_ae_vs_X0:.8f}")
print(f"SVD(k) vs true X0: {mse_svd_vs_X0:.8f}")
```

```
# 7) quick plots
plt.figure(figsize=(10,4))
plt.subplot(1,3,1); plt.title("Original X (sampled)"); plt.imshow(X, aspect='auto'); plt.col
plt.subplot(1,3,2); plt.title("AE Recon"); plt.imshow(recon_np, aspect='auto'); plt.colorbar
plt.subplot(1,3,3); plt.title("SVD Recon"); plt.imshow(X_svd_k, aspect='auto'); plt.colorbar
plt.tight_layout()
plt.show()

# 8) Print a small summary
print("\nSummary:")
print(f"Matrix size: {m} x {n}, true rank: {true_rank}, target rank: {target_rank}")
print(f"AE MSE: {mse_ae:.6e}, SVD MSE: {mse_svd:.6e}")
```

```
Epoch 001, Train MSE: 9.848120
Epoch 050, Train MSE: 0.077673
Epoch 100, Train MSE: 0.023026
Epoch 150, Train MSE: 0.022104
Epoch 200, Train MSE: 0.013661
Epoch 250, Train MSE: 0.010162
Epoch 300, Train MSE: 0.018755
Epoch 350, Train MSE: 0.030071
Epoch 400, Train MSE: 0.010699

Autoencoder reconstruction MSE: 0.01111963
NumPy truncated SVD (k=10) reconstruction MSE: 0.00222456
MSE noisy vs true X0: 0.00250672
Autoencoder vs true X0: 0.00984662
SVD(k) vs true X0: 0.00028217
```




```
Summary:
Matrix size: 200 x 150, true rank: 10, target rank: 10
AE MSE: 1.111963e-02, SVD MSE: 2.224559e-03
```